

Aperture — Backend Architecture

Version: v1.0

Document Type: Backend Architecture

Related Documents

- Technical Architecture
 - Low-Level Design
 - API Specification
 - Database Design
-

1. Purpose

This document defines how the Aperture backend is organized internally.

While the Technical Architecture explains *why* technologies were selected, this document explains *how* the backend is structured to maximize maintainability, testability, and scalability.

The architecture follows the principles of a **modular monolith**, where each business domain is isolated behind well-defined interfaces while remaining deployable as a single application.

2. Architectural Philosophy

The backend is designed around several principles:

- Domain-first organization
- Clear separation of concerns
- Explicit module boundaries
- Dependency inversion
- Testability
- Evolutionary architecture

Business domains should remain independent wherever possible.

Infrastructure should support the domains rather than define them.

3. High-Level Project Structure

```
backend/
├── app/
│   ├── auth/
│   ├── users/
│   ├── metadata/
│   ├── reviews/
│   ├── lists/
│   ├── search/
│   ├── discovery/
│   ├── streaming/
│   ├── notifications/
│   ├── admin/
│   ├── common/
│   ├── core/
│   └── main.py
├── tests/
├── migrations/
├── scripts/
├── docs/
├── docker/
└── pyproject.toml
```

The structure mirrors the business domains defined in the LLD documents.

4. Module Structure

Every business module follows a consistent internal layout.

```
reviews/
├── api.py
├── service.py
├── repository.py
├── models.py
├── schemas.py
├── dependencies.py
├── exceptions.py
└── events.py
```

```
├─ tasks.py
└─ tests/
```

Not every module will require every file initially, but the structure provides a predictable pattern as the project grows.

5. Request Lifecycle

Every incoming request follows the same path:

```
Client
↓
FastAPI Router
↓
Authentication
↓
Validation
↓
Service Layer
↓
Repository Layer
↓
Database
↓
Response Serialization
↓
Client
```

Cross-cutting concerns such as logging, metrics, and exception handling are applied through middleware rather than individual endpoints.

6. Layer Responsibilities

API Layer

Responsible for:

- Routing
- Request validation
- Response serialization
- Authentication hooks
- HTTP status codes

No business logic should live here.

Service Layer

Contains:

- Business rules
- Workflow orchestration
- Transaction boundaries
- Cross-module coordination

This is the core of the application.

Repository Layer

Responsible only for persistence.

It should:

- Query the database
- Persist entities
- Handle pagination
- Abstract SQLAlchemy interactions

Repositories should not make business decisions.

Domain Models

Represent persistent entities and relationships.

They should remain focused on the domain rather than transport concerns.

Schemas

Define API contracts using Pydantic.

Separate request and response models improve validation and API stability.

7. Dependency Injection

FastAPI's dependency injection system is used throughout the application.

Dependencies include:

- Database sessions
- Current user
- Authorization checks
- Configuration
- Shared services

Dependency injection keeps modules loosely coupled and simplifies testing by allowing dependencies to be replaced with mocks or fakes.

8. Configuration Management

Configuration is centralized.

Sources include:

- Environment variables
- `.env` files (development)
- Secret management (production)

Configuration categories:

- Database
- Authentication

- Redis
- TMDb API
- Logging
- Feature flags

Configuration should be strongly typed and validated during application startup.

9. Shared Infrastructure

The `common` and `core` packages contain reusable infrastructure.

Examples:

- Authentication utilities
- Database helpers
- Exception handling
- Logging
- Pagination
- Rate limiting
- Response utilities

Business modules should avoid duplicating shared functionality.

10. Error Handling

Errors are categorized into:

- Validation
- Authentication
- Authorization
- Business rule violations
- External service failures
- Internal errors

Exceptions should propagate to centralized handlers that produce consistent API responses.

11. Background Tasks

Long-running operations execute outside the request-response cycle.

Examples:

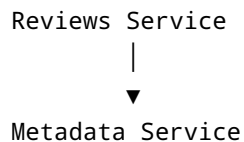
- Metadata synchronization
- Recommendation generation
- Search indexing
- Notification delivery
- Email sending

Initially, FastAPI background tasks are sufficient. As complexity grows, these workloads can migrate to a dedicated task queue without changing business logic.

12. Cross-Module Communication

Modules communicate through service interfaces rather than direct repository access.

Example:



```
graph TD;
    ReviewsService[Reviews Service] --> MetadataService[Metadata Service];
```

The diagram illustrates a dependency between two services. At the top, the text "Reviews Service" is displayed. A vertical line with a downward-pointing arrowhead connects it to the text "Metadata Service" at the bottom, indicating that the Reviews Service depends on or calls the Metadata Service.

This preserves module boundaries and prevents tight coupling.

13. Event-Oriented Design

Although Aperture begins as a modular monolith, modules communicate conceptually through domain events.

Examples:

- ReviewCreated
- RatingChanged
- UserFollowed
- ListCreated
- AvailabilityUpdated

Initially, these events are handled in-process.

The abstraction allows migration to an external message broker in the future with minimal changes.

14. Middleware

Global middleware responsibilities include:

- Request logging
- Correlation IDs
- Authentication
- Error translation
- CORS
- Security headers
- Metrics collection

Middleware should remain lightweight and reusable.

15. Transaction Management

Transactions should encompass a single business operation.

Examples:

- User registration
- Review creation
- Follow user

Long-running or external operations should occur after the primary transaction has completed.

16. Logging Strategy

Logging is structured and contextual.

Every request should include:

- Request ID
- User ID (when available)
- Endpoint
- Duration
- Status code

Sensitive information must never be written to logs.

17. Security Practices

Security measures include:

- Password hashing
- JWT validation
- Input validation
- SQL injection protection via ORM
- Rate limiting
- Least-privilege database access

Security is applied consistently across modules rather than individually.

18. Testing Integration

Each module owns its own tests.

Testing categories:

- Unit
- Integration
- API
- Performance (where appropriate)

Shared test utilities live outside business modules to avoid duplication.

19. Operational Considerations

The backend should expose:

- Health endpoints
- Readiness checks
- Liveness checks
- Version information

Operational endpoints remain separate from business APIs.

20. Future Evolution

The architecture is intentionally designed to evolve.

Potential future changes include:

- Service extraction for Discovery or Notifications
- External message broker
- Dedicated recommendation service
- Independent search infrastructure

Because module boundaries are enforced from the beginning, these changes should not require significant refactoring of existing business logic.

21. Interview Discussion Points

Be prepared to explain:

- Why choose a modular monolith over microservices?
 - Why separate API, Service, and Repository layers?
 - How does dependency injection improve testability?
 - Why communicate between modules through services instead of repositories?
 - How would this architecture evolve into independently deployable services?
 - What belongs in middleware versus business logic?
-

22. Summary

The Aperture backend is organized around stable business domains with clear architectural boundaries.

By combining a modular monolith, layered architecture, dependency injection, and event-oriented communication, the codebase remains approachable for a single developer while providing a realistic migration path toward larger-scale production systems.

The goal is not simply to build a working backend, but to build one that is easy to understand, maintain, test, and evolve.